

CODE APHA ML INTERNSHIP

Stages of System Development: Character Recognition (MNIST)

Choosing and Obtaining the Dataset

For Task 4 (Character Recognition), we make use of a Neural Network / Convolutional Neural Network (CNN) approach or dense Neural Network architecture using Keras / TensorFlow.

- **Dataset Selection:** We utilize the MNIST Handwritten Digit Dataset, which consists of 70,000 grayscale images of digits (0 through 9) formatted as 28x28 pixel grids.
- **Obtaining Data:** The dataset is imported directly via `keras.datasets.mnist` or downloaded as CSV/Numpy array files into the project directory.

2. Understanding and Normalizing the Input Data

The raw dataset consists of digit images where each pixel value ranges from 0 to 255 (0 representing black, 255 representing white, and values in between representing shades of gray).

- **Features (X):** The pixel intensity matrices representing image samples.
- **Target (y):** Categorical ground-truth labels indicating digits from 0 to 9.
- **Reshaping:** Images are reshaped into $(-1, 28, 28, 1)$ or flattened into a 1D vector of length 784 ($28 * 28$), depending on the layer architecture.
- **Data Normalization:** Pixel values are divided by 255.0 to rescale feature values to a floating-point range between 0.0 and 1.0. This normalization helps gradient descent converge faster during model training.

Data Separation and Preprocessing

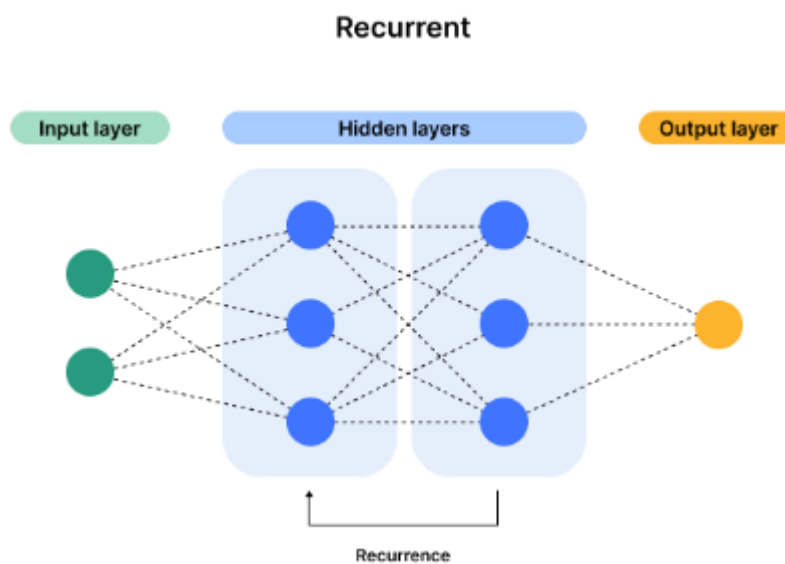
Before feeding data into the neural network, we split and preprocess the dataset:

- **Train / Test Split:** Data is split into Training Set (typically 80% or 60,000 samples) and Testing Set (typically 20% or 10,000 samples).
- **Label Encoding:** Labels (0–9) are converted to one-hot encoded vectors or handled via Sparse Categorical Crossentropy to align with the final activation layer.

🚦 4. Neural Network Architecture & Training

We construct a multi-layer neural network architecture consisting of:

1. **Input / Flatten Layer:** Converts the 28x28 image grid into a 1D line of 784 pixels.
2. **Hidden Layer(s):** Dense layers with Activation Functions (such as ReLU) where deep feature extraction and learning occur.
3. **Output Layer:** A Dense layer with 10 nodes (representing digits 0–9) paired with a **Softmax** activation function to output class probabilities for each digit.



🚦 Model Evaluation and Performance Tuning

Initial Evaluation: Model accuracy and loss are evaluated on the test set.

Hyperparameter Tuning: Adjustments to batch size, epochs, number of hidden layer units, and learning rate are performed to maximize validation accuracy.

🚦 Saving, Exporting, and Deployment

- **Saving Model:** The trained model is exported as a .keras or .h5 file using Keras saving utilities to preserve architecture, weights, and optimizer configuration.
- **Deployment:** Integrated with a backend service (e.g., FastAPI) to connect with a web interface (HTML/JS or React) for real-time digit prediction.